ChatGPT

Palantir Ontology 如何運作與顧問工作分析系統承接設計報告

執行摘要

Palantir 的 Ontology（在 Foundry/Gotham 語境下）並非傳統意義「只讀」的知識圖譜或資料字典，而是一個**可被應用與工作流直接運行的營運層（operational layer）語意模型**：它把組織已整合的數位資產（datasets、virtual tables、models）映射為真實世界的業務「名詞」（objects / properties / links），並補上「動詞」（actions / functions）與「動態治理」（dynamic security、細粒度權限、審計）以支持可控寫回與決策閉環。 [1]

若你的目標是把 Ontology「完整承接」成顧問工作分析系統的底座，最關鍵不是複製其 UI 或特定資料庫，而是複製它的**架構原則**：
其一，**語意層與可執行層合一**（資料模型 + 可控寫回 + 可重用邏輯 + 權限/稽核），使分析結果能回寫成可追溯的決策與行動；其二，**一切互動以 Object 為中心**（Object Sets、Search Around、Object Views / Workshop），讓不同規模、不同領域的問題都能在同一套「名詞/動詞」框架下被操作；其三，**內建變更管理與版本治理**（Global Branching、提案/審核、schema migration），讓模型能在制度化與規模化階段持續演進而不破壞既有應用。 [2]

在落地建議上，若你**有** Foundry/AIP 授權且可在其上開發，最短路徑是：以 Foundry Ontology + OSDK（TypeScript/Python/Java）做你的顧問系統後端，將「顧問輸入→抽取→對齊→風險/策略→行動清單→審核→寫回」整個閉環做成可重用的 Ontology 物件與 Action/Function。 [3]
若你**沒有**或不希望被供應商鎖定，則以「**Operational Ontology**」原則自建：採用圖譜/文件/向量/事件稽核的混合式架構，並把 Palantir 的幾個關鍵能力（Object/Link、Action、Object Set、Branching、Audit、語意檢索）以開源與標準工具重建；缺點是工程與治理成本顯著上升。 [4]

本報告同時標註：Ontology 後端的部分實作細節（例如某些索引資料結構、Entity Resolution 內建演算法參數、部分 AIP agent 內部編排）屬於**未公開**；對於這些部分，我提供合理推測與可替代方案（以公開文件可驗證的介面行為/限制為邊界）。 [5]

官方與原始技術來源

下表整理本報告主要依據的「官方/原始」資料（以 Palantir 官方文件、官方 API Reference、官方開源 SDK 為主；少量中文資料僅作輔助理解、且明確視為二手解讀）。

主題	來源（點選引用即為連結）	為何關鍵
Ontology 定位與核心概念	Foundry Ontology Overview、Core concepts、Why create an Ontology? [6]	定義 Ontology 是「operational layer / digital twin」，並明確區分 semantic elements 與 kinetic elements。 [7]

主題	來源（點選引用即為連結）	為何關鍵
Ontology 資源構成與命名空間	Ontologies overview（ontology 與 space 1:1、shared ontologies）、Object type / Property / Link metadata reference、API naming guidelines [8]	釐清本體的「schema 資源」範圍（object/link/action/interface/shared property…）與 API name/RID/status 等治理欄位。[9]
多型、層級與語意約束	Interfaces overview / metadata / implement / extend、Interface link type constraints、Required properties、Mandatory control properties [10]	這組文件是「繼承/多型/約束」在 Palantir Ontology 的主要公開機制（以 interface composition/implementation 取代傳統 class inheritance）。[11]
後端架構與資料流	Object backend overview（OMS/OSS/Actions/Funnel/OSv1 vs OSv2）、Indexing overview、Funnel batch/streaming pipelines、How user edits are applied [12]	用來還原「接入→索引→查詢→寫回→持久化→版本一致性」的實際運作。[13]
版本管理與變更治理	Test changes in ontology（Global Branching）、Foundry Branching supported functionality、Schema migrations、Ontologies migration [14]	指出 Ontology 如何在不打斷 production 的情況下演進，並處理 breaking schema change 與 edits 遷移。[15]
效能/成本/限制	Ontology query compute、Ontology indexing compute、Ontology volume、Action scale limits、Indexing FAQ（streaming latency/throughput）[16]	直接給出查詢/索引/寫回的成本模型與硬限制，對「規模化」設計尤其重要。[17]
API/SDK（可落地實作）	Foundry API Introduction（REST/OAuth2）、OSDK overview & subscriptions、Foundry Platform SDK（PyPI）、官方開源 SDK：foundry-platform-python、foundry-platform-typescript、osdk-ts [18]	這是把 Ontology 變成你系統底座時，最可直接複用的「介面契約」。[19]
Ontology-aware 應用層整合	Ontology-aware applications、Workshop overview、Object Explorer overview、Object Views overview、Quiver overview [20]	解釋 Ontology 如何直接驅動 UI/工作流/分析。[21]
Gotham 整合與 Entity Resolution	Gotham API intro（RevDB & dynamic ontology）、Enable Gotham integration、Create Ontology objects from Gaia、Cross-app interactivity（Foundry↔Gotham drag&drop enrichment）、Gotham object resolution basics [22]	提供「跨平台 object 同義映射」與「合併物件（resolution）」的公開定義與入口。[23]
AIP 與語意檢索（RAG）	AIP overview、AI ethics & governance、AI FDE security/governance、Ontology-augmented generation（semantic search、HyDE、query augmentation、hybrid search）[24]	直接描述 Palantir 如何把文件切塊成 objects、用 embedding/vector property 做檢索，並把 AI 動作納入權限與稽核。[25]
（二手中文輔助）	中文快讀（非官方）：Foundry Ontology 介紹文章（繁中）[26]	可作概念補強，但不建議用作規格依據。[26]

補充：Foundry API 的完整 OpenAPI 檔「是否可下載」在 2025 年官方社群回覆為「當時不可、但在規劃中」，屬於**官方介面可用、但規格檔對外供給曾受限**的典型例子；因此本報告對「可實作細節」以 API Reference 與官方 SDK（自動生成的 endpoint 列表）為準。 [27]

Ontology 概念模型與語意治理

核心概念與資源邊界

在 Foundry 的官方定義裡，Ontology 是組織的營運層，位於平台已整合的數位資產之上，並把它們連回真實世界對應物（資產、訂單、交易等）；在許多情境它被描述為組織的 digital twin，包含 semantic elements（objects, properties, links）與 kinetic elements（actions, functions, dynamic security）。 [28]

官方文件進一步把「ontology」當成一種 artifact，裡面存放的 Ontology resources 明確包含：Object types、Link types、Action types、Interfaces、Shared properties、Object type groups 等。 [29]

Ontology 與 Foundry 的「Space」是 1:1 對應：建立新 space 時會同時建立同名 ontology，且繼承相同的 organization markings；private/shared space 對應到 private/shared ontology，並支援多組織共享資料與工作流（shared ontologies）。 [30]
這個設計對你的顧問系統很重要：它揭示 Palantir 的「命名空間」不是靠 RDF namespace 或資料庫 schema，而是靠 **space/ontology 作為安全與協作的隔離單位**。 [31]

Object types、Properties、Links 的語意層

Object type 的官方說法是：Ontology 不是抽象模型，而是把概念映射到組織真實資料；要讓某 object type（例如 Employee）產生 objects，會在 Ontology Manager 中替該 type 加入 backing datasources，並把企業資料（例如員工目錄）接進 Ontology。 [32]

Object type 的 metadata（治理欄位）包含：ID、RID、API name、visibility（prominent/normal/hidden）、status（active/experimental/deprecated）、index status、writeback 是否啟用等。 [33]
Properties metadata 亦包含 RID、status、title key / primary key、base type、render hints（影響索引/排序/搜尋與 reindex 表現）等。 [34]
Link type 同樣有 API name 並可在兩端定義用於程式存取的名稱（例如呼叫 `Company.employee.get()` 取得 linked Employee objects 的例子）。 [35]

Property base types 在 Foundry 中不僅是基本型別，還包含一些「為 operational workflow 準備」的進階型別：Vector（semantic search）、Geopoint/Geoshape、Attachment、Time series、Media reference、Struct types 等。 [36]
官方也明示 Foundry data types 的概念受 RDF/OWL/XSD 啟發（但 Foundry 並非等同 RDF/OWL 的推理型本體）。 [37]

層級、繼承、多型與語意約束

Palantir 對「多型」的主要公開機制是 **Interfaces**：Interface 描述 object type 的 shape 與 capabilities，可由多個 object types 實作，並提供 polymorphism 讓共享 shape 的 objects 能被一致地建模與互動（例如 Facility interface 由 Airport/Plant 等實作）。 [38]
Interface 由 interface properties、link type constraints、metadata 構成；properties 可在 interface 上本地定義（官方建議）或使用 shared properties；且 object types 可實作多個 interfaces 以支援不同工作流。 [39]

Interface 也支援「組合式繼承」：extend interface 可讓新 interface 繼承被 extend 的 shared properties 與 links，且一個 interface 可 extend 多個 interfaces（多重組合）。 [40]

這非常適合你要橫跨創業→制度化→規模化、跨領域（營運/財務/法務/行銷/業務/自媒體/品牌）的需求：你可以把「成熟度階段」與「領域能力」都做成 capability interfaces，再由不同規模/產業的 object types 去實作，得到可擴充且不破壞既有模型的多型結構。 [40]

語意約束方面，公開文件至少包含三類可直接用於「資料品質 + 工作流可預測性」的約束：

其一，**Required properties**：可設定某 property 必須有值，會對 backing datasource 的資料與 action edits 都做驗證；若現有資料含 null，reindex 會失敗，必須清理來源或取消 required。 [41]

其二，**Interface link type constraints**：可在 interface 上定義跨實作 type 的共同關係（target type 可是 interface 或 object type、可設定 1:1 / 1:many、可 required/optional），實作時由具體 link types 去 fulfill constraint。 [42]

其三，**Mandatory control properties（OSv2）**：用 marking-backed restricted view 與 marking column，讓某一控制欄位的權限要求「門控」同一 datasource 中其他 properties 的可見性，達成更細粒度的 property access control；而且 mandatory control 本身必須 required，並對 allowed values 有限制與 enforcement。 [43]

命名、命名空間與版本標記

Ontology 資源在平台內部同時有 **RID**（唯一識別、常出現在錯誤與平台訊息中）與 **API name**（程式存取的穩定名稱）。 [44]
官方也提供 API naming guideline：object type API name 需 PascalCase、property API name 需 camelCase、需唯一、長度限制等。 [45]
這些設計對你自建系統的啟示是：若你想要「生成 SDK、讓前端/自動化依賴 schema」類 OSDK 體驗，你必須把 API name 當成 public contract，並提供和 RID 分離的「人類可讀穩定名稱」。 [46]

資源的 version/生命週期標記則見於 status（active/experimental/deprecated）等欄位，並會影響使用者與其他建模者如何判斷資源成熟度。 [47]

資料流、索引查詢與更新機制

從資料接入到 Ontology 物件的生成

Foundry 的 Ontology 以「把 datasources 索引成 objects」為核心路徑：Object backend 的職責分成三大類——datasource management、query/search/aggregate、以及寫回與索引編排。 [48]
在 OSv2 架構中，**Object Data Funnel（Funnel）**負責把 Foundry datasources（datasets、restricted views、streaming datasources）與 Actions 產生的 user edits 一起編排並索引到 object databases，並確保底層資料更新時索引保持最新。 [49]

在接入層，Pipeline Builder 的 outputs 可直接產生 datasets、virtual tables，或 Ontology components（object types、object links、time series 等），使「清洗/建模/索引」能在同一條 pipeline 中串接。 [50]
對非結構化資料，transforms Python 提供 FileSystem 介面讀寫 dataset 內檔案，支援把 PDF、影像等處理後轉成可索引的結構化輸出（例如切塊、抽取、embedding）。 [51]

索引、查詢與 Object Sets

Object backend 中的 **Object Set Service（OSS）**負責 serving reads，支援 searching/filtering/aggregating/loading。 [52]
Object sets 是跨 Foundry 應用共享的核心容器：可分 static（儲存 primary keys 列表）與 dynamic（儲存

filters 定義，資料更新後結果會變），並分 temporary（僅創建者可用、24 小時過期）與 permanent（可長期保存）。 [52]

這種設計對顧問系統非常「對味」：你可以用 object set 表達「這次診斷範圍內的客戶/合約/流程/風險集合」，再把同一集合丟進策略函數、報告模板、儀表板與寫回 action，形成可重放的分析與執行單位。 [53]

寫回、審計、版本一致性與衝突解決

Palantir 把可控寫回放在 Ontology 的「kinetic」層：Actions service 以結構化方式修改 object databases（包含權限與條件），也可形成可分析的 action log 以研究使用者決策。 [54]

在 OSv2 的公開行為描述中，當 action 觸發後 Actions service 會把 modification instruction 發送到 Funnel，該 instruction 放入具 offset tracking 的 queue 以支援同時編輯；而且一旦修改送出，後續 ontology query 的 object read **保證包含 user edits**。 [55]
user edits 會立即應用到索引並週期性 flush 到 Funnel 管理的 persistent datasets（官方稱 persistent store 以 dataset 形式存在）。 [56]

「撤銷」方面，官方明示沒有直接 undo 單一 edit 的機制，只能再做 additional edits；若要重置，可用 OSv2 schema migration framework 的 “drop all edits” 指令刪除該 object type 的既有 edits。 [57]

版本一致性上，官方文件把問題拆成兩層：前端載入版本 vs actions server 執行時載入版本可能不一致，因此 Ontology 提供 object types 與 object instances 的 version consistency 機制；在 OSv1 會檢查版本並可能丟 `StaleObject` error。 [58]
衝突解決策略（如「比 timestamp、採用最新值」）是在 OSv2、且以 object type 粒度設定。 [59]

索引限制、資料限制與效能成本模型

Ontology indexing 的 compute 模型公開指出：索引使用 parallelized Spark 後端從大規模資料讀取並轉成 Ontology 格式；compute 使用取決於 driver/executor 資源與 wall-clock 時間，並可在 Builds 中追蹤 indexing jobs。 [60]
Ontology volume 則以「indexes 的總大小」計量，且提示可能大於 dataset volume（因 Ontology data 無法壓縮、索引需額外儲存以加速查詢）。 [61]

查詢成本方面，官方給出 OSv1/OSv2 的固定 compute-second overhead（OSv1 每 query 固定 16、OSv2 固定 4），並指出 search-arounds、writeback 超過一定規模可能改用 on-demand Spark execution layer；Actions 與 Functions on Objects 也各有固定 overhead（例如 action 最小 18、function execution 固定 4 等）。 [62]

動作規模限制也有明確數值：單次 action submission 最多可編輯 10,000 objects、可涉及最多 50 object types、參數 list 長度上限等；且單一 object edit 的 payload size 在 OSv2 可到 3MB（OSv1 為 32KB）。 [63]

對 streaming ingestion，OSv2 透過 Funnel streaming pipelines 支援秒/分鐘級低延遲索引；FAQ 說明「exactly once」一致性主要成本來自 Flink checkpointing（預設每秒一次），並給出預期 throughput（每 object type 2MB/s）、不可 partial row 更新、以及保序索引等行為。 [64]

此外，OSv2 會在 indexing 時做 data restrictions 驗證：batch datasource 違規會造成 indexing job fail；streaming datasource 違規 record 會被丟棄。 [65]
這可被視為「語意層的資料品質門檻」——對顧問系統來說等價於：你若要把分析結論寫回成可行動資料，你必須有類似的 validation 與 error governance，否則會讓工作流在規模化階段失控。 [66]

實體解析（Entity Resolution）

公開文件中，Gotham 對「object resolution」的定義非常直接：把兩個或多個 objects 合併，用於避免不同 source systems 指向同一實體卻產生重複 objects。[67]
Foundry 也提供 Entity Resolution 產品頁面，強調涵蓋 fuzzy matching、feature generation、model training、human validation 等 pipeline tooling。[68]

未公開：Entity Resolution 的內建模型/演算法細節（特徵工程預設、訓練/評估指標、blocking/association 的策略、active learning 迴圈等）在官方公開文件層級並沒有完整展開；因此本報告不把它當作可直接照抄的規格，只把它視為「平台提供端到端 ER workflow」的能力宣告。[68]
替代方案方面，Foundry 文件與社群討論顯示可用 transforms 的 fuzzy matching 範例（phonetic codes、Jaro similarity 等）、或在 Pipeline Builder 用 Levenshtein distance 與 LLM node 做近似匹配（視情境）。[69]

與 Foundry、Gotham、AIP 的整合點與 API/SDK 實作

Ontology-aware 應用層：從資料到操作介面的「直連」

Foundry 明確列出多個「原生在 Ontology 上運作」的應用，並強調它們共同構成 analytical + operational 平台；其中 Object Views 是以單一 object 為中心的資訊與工作流樞紐，可包含 “biographical data”、linked objects、嵌入的 workflow/分析/儀表板等。[70]
Object Explorer 則是面向較少技術背景使用者的搜尋與分析工具，支援從 keyword search 到 property filters、探索 view/表格 view、bulk actions、export、並可把 object set 打開到 Quiver 等其他應用。[71]
Workshop 作為 operational app builder，官方明示其資料讀取以 Object layer 為主、寫回透過 Actions、商業邏輯透過 Functions，並以 layouts/events 提供接近自訂前端的互動能力。[72]
Quiver 則提供點選式 object/time series 分析，支援沿 link type 導航並嵌入 Workshop/Notepad。[73]

這一套「Object → View/Explore → Action/Function」的組合，是你要做顧問系統時最值得承接的互動原型：使用者不是在填表或查報表，而是在操作「客戶公司、產品線、流程、合約、風險」等 objects，並以 actions 觸發可審計的變更。[74]

Foundry API（REST/OAuth2）與 Ontologies v2 介面輪廓

Foundry API 的公開定義為 developer-friendly REST API，使用 OAuth 2.0 驗證，主要以 JSON request/response 的一致模式設計。[75]
在 Python 的 foundry-platform-sdk（官方自動生成 SDK）中，可見 OntologiesV2 相關 endpoints，包含：objectTypes/list/get、objects/get/list/page/search、linked objects、queryTypes/list/get、queries/execute、actions/apply/applyBatch、attachments、timeseries points 等。[76]
這些 endpoint 形狀，對你做「顧問系統中介層」很關鍵：就算你不在 Palantir 上開發，也可以把它當作「Operational Ontology API 的參考藍圖」。[77]

同時，foundry-platform-python（官方 SDK repo）也直接說明：Foundry/Gotham 各有 platform API 與 SDK；另外還有 OSDK 用於 Foundry ontologies，且一般建議「凡是 leverage Ontology 的應用，優先用 OSDK」。[78]

未公開/不穩定處：Foundry API 的「完整 OpenAPI 規格檔」是否可直接下載，官方社群在 2025 年回覆曾為「當時不可、但在規劃中」；然而，foundry-platform-sdk 本身又宣稱其由 OpenAPI spec 自動生成並連到 openapi.yml。兩者可以調和為：規格檔存在且用於生成 SDK，但其對外「完整可下載/可匯入 Postman」的交付方式曾受限或在演進中。[79]

OSDK（Ontology SDK）：把 Ontology 變成「可型別安全開發的後端」

OSDK 官方定位是「讓你在開發環境中直接存取 Ontology」，支援 TypeScript（NPM）、Python（pip/conda）、Java（Maven），以及以 OpenAPI spec 支援其他語言；並強調可把 Foundry 當後端，利用其高規模 query + writeback + granular governance 來加速安全開發。 [80]
OSDK 的 security model 也公開指出：token scope 會限制到特定 ontological entities，並仍受使用者自身對資料的權限限制。 [81]
TypeScript OSDK 也提供 subscriptions，可對指定 object set 的變更（新增/刪除/屬性變更）接收即時更新。 [82]

你要承接到顧問系統時，可把 OSDK 視為「以 schema 生成 client」的範本：schema 的 API names/metadata 會直接進入程式體驗，形成工程效率與一致性。 [46]

Gotham 整合、跨應用互動與同義映射

Gotham API 同樣是 REST/OAuth2，並明示其目前支援 Gotham RevDB（object database backed by a dynamic ontology）。 [83]
Foundry↔Gotham 的整合在文件中主要透過「type mapping / synonyms」與 cross-application interactivity（drag-and-drop、app pairing）呈現：若同一真實世界物件在 Foundry 與 Gotham 各有表示，可被視為 synonyms，透過 type mapping 建立對應，並在 drag-and-drop 時用 data enrichment 發現同義表示以增加互通性。 [84]

啟用 Gotham integration 的公開流程包含：在 Ontology Manager 為 object type 開啟 type mapping；若 enrollment 有 Map Rendering Service（MRS），可在 Gaia 直接建立/編輯 Ontology objects（部分功能依賴 MRS）。 [85]

審計與治理的整合點

Foundry 安全總覽描述平台具備強制加密、強驗證、強授權（mandatory/discretionary access controls）、稽核記錄、可擴充治理與隱私控制等。 [86]
Audit logs 文件則明示它提供對平台「每個 action」的綜合紀錄，用於威脅偵測、事故調查、合規與問責。 [87]

在 AI 與 agent 的脈絡下，AI FDE 文件更進一步指出：AI FDE 不是 service account，而是以你的 session/identity 執行，所有行為受相同 permission checks、governance controls、audit logging；對 mutating operations 有 user approval 機制；所有 API call 都帶你的 identity，並可在標準 audit logs 中追溯。 [88]
AIP ethics & governance 也把「Ontology-based decision support + actions/approval workflows + audit trails」寫成負責任 AI 的核心實作路徑之一。 [89]

顧問分析系統的具體應用設計

本節把 Palantir Ontology 的公開機制（Object/Link、Interface、多型、Action/Function、Object Set、Semantic search、Branching/Audit）映射成一套可用於顧問工作、可跨產業與規模的系統設計；其中屬於你系統內部的設計選擇，我會以「建議/可選方案」呈現，而非宣稱 Palantir 既有功能。 [90]

顧問 Ontology 的建議概念模型

建議你把顧問系統拆成「名詞（Objects）」與「動詞（Actions/Functions）」，並以 interfaces 做跨領域共用能力。這正符合 Palantir 對 semantic/kinetic elements 的分層語言，也能最大化日後規模化時的重用。 [91]

核心 Object types（跨領域最小集合，MVP 建議先做）

- Organization（組織/事業體）、BusinessUnit、Role/Person（若需）、Product/Service、Channel、Market/Segment
- Process（流程）、Policy/Control（制度/控制）、Contract/Clause/Obligation（法務）、Asset（資產）、Metric（指標）、CashFlowItem（財務）
- Initiative（專案/策略）、Task（任務/待辦）、Decision（決策）、Risk（風險）、Assumption（假設）、Evidence（證據：文件片段/數據點）

Interfaces（多型與擴充策略）

- MaturityStage interface：StartupLike / Institutionalizing / Scaling（每個 stage 定義必備屬性與常用 links，如「現金跑道」「標準作業程序」「內控」等）——用 extend interface 做 stage 組合。 [40]
- DomainCapability interfaces：OpsCapability、FinanceCapability、LegalCapability、MarketingCapability、SalesCapability、MediaCapability、BrandCapability
- RiskInterface：任何會被風險引擎處理的 object type 都實作（讓 Risk 可以 link 到不同 domain objects，類似 Palantir 用 interface link type constraints 連結多種 facilities/alerts 的方式）。 [92]

語意約束（避免顧問系統「漂亮但不可操作」）

- Required properties：例如 Risk 必須有 `impact`、`likelihood`、`owner`、`status`；Contract 必須有 `effectiveDate`、`counterparty` 等，並把它當資料品質 gate（同 Palantir required properties 會影響 reindex/edits 的邏輯）。 [41]
- Mandatory control properties（對應你的系統可做 ABAC/Marking 權限門控）：例如 Legal/HR 類資料以「標記」控制其他欄位可見性，避免顧問報告/AI 摘要外洩敏感欄位。 [93]

把輸入映射到 Ontology 的流程

你提出的輸入形式涵蓋「一句話、少量文件、或大量文件/連結」。建議用同一條管線，但依資料量調整抽取策略與人審節點（這與 Palantir 將 AI 置於權限與審核流程內的精神一致）。 [94]

輸入類型：一句話（最小上下文）
建議策略：

- 用 LLM 做「任務意圖 + 初步實體抽取」：辨識這句話在講哪個 Organization/產品/指標/風險，並產生候選 objects 與 links。
- 以 entity resolution（字串相似/規則/embedding）對齊既有 objects，避免重複建立（可參考 Gotham object resolution 概念：合併多個 objects 避免 duplicates）。 [95]
- 產生 “Intake” object（一次顧問需求/事件），把該句話作為 Evidence 並鏈接到初步 objects，後續所有分析與決策都掛在這個 Intake 上（利於審計與可追溯）。 [96]

輸入類型：少量文件（例如 1–10 份 PDF/簡報/合約）
建議策略：

- 把文件切塊（chunking），建立「Chunk objects」並用 media reference property 指向原文件，這與 Palantir 的 ontology-augmented generation 建議一致。 [97]
- 為 chunk 建立 embeddings（可用你自選模型），並存入 vector property（Foundry 也把 Vector 列為 base type 用於 semantic search）。 [98]
- 以 hybrid 檢索（keyword relevance + vector similarity）與 query augmentation（例如 HyDE、query extraction）改善召回，這是 Palantir 在同頁面列出的典型做法。 [99]

- 以抽取器（LLM/規則/表格解析）把合約條款、KPI、風險敘述等映射成「可操作 objects」（ContractClause、Obligation、Metric、Risk），並用 links 回指證據 chunk（Evidence）。 100

輸入類型：大量文件/連結（例如資料室、Notion、網站、社群、CRM 匯出）
建議策略：

- 先建立「Corpus」層的資料治理（來源、授權、敏感度、更新頻率），做成 objects，並把抓取/同步當成可審計的 Actions（避免黑盒爬蟲）。 101
- 大量資料時，需更重視索引成本與 volume：Palantir 明確把 Ontology volume 與 indexing compute 當資源管理項，並指出 volume 與 objects/props/links 的數量與大小相關。你的系統同樣需要「限制胖字段」與「分層存放」策略（例如長文留在文件索引，於 Ontology 中僅保留可操作摘要與 pointer）。 102

自動補足背景：外部搜尋與模型整合點

你希望「自動補足背景（含利用外部搜尋/語言模型）」。在 Palantir 的公開方法論中，重點不在「直接讓 LLM 回答」，而在先找對 context：ontology-augmented generation 文件明言設計 RAG 時「找到 relevant context」通常是最難的步驟，並列出 chunking、embedding、hybrid search、query augmentation 等路徑。 103

對你的顧問系統，我建議把背景補足拆成三段（每段都可審計、可回放）：1) **內部證據檢索（你自己的 corpus）**：以 chunk objects + vector/keyword 索引為主。 104
2) **外部公開資訊檢索（web search）**：只把「來源 URL + 摘要 + 引用片段」存為 Evidence objects，並加上版權/授權標記（避免把全文抄進知識庫）。這屬於你系統設計，但其治理精神與 Palantir 強調 audit trail/責任鏈一致。 105
3) **模型推論（LLM）**：要求模型輸出「結論 + 證據引用（指向 Evidence objects）+ 不確定性/缺口」，並把推論本身作為 Decision draft，進入 human oversight/approval action。 94

風險預知、策略建議與執行方案的可量化輸出格式

Palantir 公開文件把「可審計責任鏈」與「人為監督」視為 AI/自動化的核心，因此你系統的輸出不應只是自然語言建議，而應是**可寫回、可追蹤、可優先排序**的 objects。 106

建議你定義以下量化輸出（可作為 objects 的 properties，並支援 dashboard/排序/篩選）：

- RiskScore：0–100（或 0–1），並拆成 impact/likelihood/urgency/confidence
- Priority：P0–P3（或 1–5），並有依據（evidenceRefs）
- ActionList：每個 action 含 owner、所需資源（人天/預算區間）、依賴關係、預估工期、驗證指標（KPI/OKR）
- Timeline：里程碑日期與 gating criteria（完成定義）
- AuditFields：產生者（人/agent）、時間、使用的 evidence、模型版本、審核狀態（draft/approved/applied）

以下提供一個**輸出 JSON 參考格式**（你可依技術棧改成資料庫 schema；此為建議、非 Palantir 既有規格）：

```
{
 "intakeId": "INT-2026-000123",
 "org": {"id": "ORG-ACME", "name": "Acme"},
 "risks": [
  {
   "riskId": "RISK-001",
```

```
    "domain": "finance",
    "title": "現金跑道不足",
    "score": 87,
    "impact": 0.9,
    "likelihood": 0.8,
    "confidence": 0.7,
    "evidenceRefs": ["CHUNK-778", "METRIC-CASH-01"],
    "recommendedActions": ["ACT-101", "ACT-102"]
   }
  ],
  "actions": [
   {
    "actionId": "ACT-101",
    "title": "30 天內完成成本結構重算與砍支出方案",
    "priority": "P0",
    "owner": "ROLE-CFO",
    "resources": {"peopleDays": 8, "budgetRangeTWD": [0, 50000]},
    "timelineDays": 30,
    "successMetrics": [{"metricId": "METRIC-RUNWAY", "target": ">= 9 months"}],
    "dependencies": [],
    "requiresApproval": true
   }
  ]
 }
```


未指定假設的對應做法：因你未指定預算/市場/技術棧/資料量/合規要求，你應在輸出中加入「assumptions」區塊，並把每個策略/風險的分數與優先順序明確綁定 assumptions，否則輸出很難審計與迭代（這也對齊 NIST AI RMF 強調的 risk/context 管理）。 107

系統架構建議與實作里程碑

參考架構原則

若你要「承接 Palantir Ontology 的運作方式」，架構上至少要滿足四個非功能需求：

- **可演進的 schema 與版本治理**（branch/proposal/merge、schema migration） 15
- **可控寫回與審計**（actions、approval、immutable audit log） 108
- **可擴展檢索**（keyword + vector + 連結 traversal） 109
- **細粒度權限與隱私**（ABAC/marking、property-level gate） 110

資料庫/圖譜選型與查詢語言

你要求比較 Neo4j/JanusGraph/OWL/RDF/GraphDB/Metaphactory。以下先給選型邏輯（後節有表格）：

- 若你要快速做「顧問工作流型」應用、以 traversal/關聯查詢為主：偏 **Property Graph**（例如 Neo4j 111，Cypher）。Neo4j 文件明確描述其 property graph 模型由 nodes/relationships/properties 組成，並以 Cypher 作為圖查詢語言。 112
- 若你要分散式擴展到極大圖規模：可看 JanusGraph（以 Apache TinkerPop/Gremlin traversal 為主），其 GitHub README 強調可在多機叢集上支援 billions of vertices/edges 與高並發 transaction；Gremlin 則是 TinkerPop 的 traversal language。 113

- 若你需要形式化語意、推理（RDFS/OWL）、標準交換（RDF/SPARQL）：走 RDF/OWL 生態（W3C 規範 + triplestore）。W3C 對 RDF 的定義是以 subject–predicate–object 的抽象資料模型表達資訊；OWL 2 則提供 classes/properties/individuals 等本體語言能力。 [114]
- 若你希望「RDF 推理 + 企業級工具」：GraphDB 與 metaphactory 常見組合。GraphDB 文件描述其支持 RDF/SPARQL，並以 forward chaining/total materialization 做推理；metaphactory 相關論文/文件則描述其提供可客製 UI 元件、支援知識圖譜管理與快速建應用。 [115]

索引策略（文字、關聯、時間序列）

承接 Palantir 的做法，你應區分三種索引：

- **結構化/關聯索引**：以 graph DB 的原生索引（node id、label/type、key properties）+ traversal。 [116]
- **全文索引**：對 Documents/Chunks 的內容做 keyword index；Palantir Object Explorer 也提供 fuzzy search 語法（例如 `~`），顯示其對文字檢索體驗的重視。 [117]
- **向量索引**：把 chunk embeddings 存在 vector property 或外部 vector DB，並用 hybrid search；這與 Palantir semantic search 文件所述一致。 [104]

Mermaid 系統架構圖（建議版）

```
flowchart LR
  subgraph Ingest[資料接入層]
    A1[一句話/表單輸入] --> B[Intake Service]
    A2[少量文件(PDF/Doc)] --> C[Document Processor]
    A3[大量文件/連結/資料室] --> D[Connector + Crawler]
    A4[SaaS/DB/CSV] --> E[ETL/ELT Pipelines]
  end

  subgraph Process[抽取與對齊]
    C --> F[Chunking + Metadata]
    D --> F
    F --> G[Embedding/Vectorization]
    F --> H[LLM/規則抽取: entities/metrics/clauses]
    H --> I[Entity Resolution & Canonicalization]
  end

  subgraph Ontology[Operational Ontology 層]
    J[(Graph Store\n(Property Graph 或 RDF))]
    K[(Document Store / Object Storage)]
    L[(Search Index\n(keyword + vector))]
    M[(Event/Audit Log\nappend-only)]
  end

  subgraph Serve[服務層]
    N[Ontology API\nobjects/links/object sets/queries]
    O[Action Service\nwriteback + approvals]
    P[Function/Rule Engine\nscoring/derivations]
    Q[Policy Engine\nABAC/Marking]
  end
```

```
  subgraph AI[AI 層]
    R[RAG Orchestrator\n(檢索→推論→引用)]
    S[Risk Scoring Model]
    T[Planning/Execution Agent]
  end

  subgraph UI[可視化/互動層]
    U[Object Explorer/Graph View]
    V[Dashboard/Report Builder]
    W[Workflow UI\n(inbox/tasks/approvals)]
  end

  B --> H
  E --> I
  G --> L
  I --> J
  F --> K
  H --> J
  J --> N
  K --> N
  L --> R
  N --> R
  R --> S
  S --> P
  P --> O
  O --> M
  Q --> N
  Q --> O
  N --> U
  N --> V
  O --> W
  R --> V
```

此圖中的「objects/links/object sets/actions/functions + governance/audit + semantic search」對應到 Palantir 公開架構中的 OMS/OSS/Actions/Funnel、Object sets、AIP 的 ontology-based workflows 與 audit/approval。 [118]

實作步驟與里程碑（含 MVP）

以下以「未指定」假設下的**三種規模**估算：

- 小型：1–2 位工程（偏原型）
- 中型：3–5 位工程（可上線 pilot）
- 企業型：6–10 位工程（含合規/資安/高可用）

表格時間以「週」估算（可並行項目以最短路徑為準）：

里程碑	交付物	驗證指標（例）	人力/時間估算（小/中/企）	主要風險與緩解
Ontology MVP 定義	核心 object types、interfaces、命名規範、最小 links、required properties	能覆蓋 ≥10 個顧問常見問題（現金跑道、漏斗、合約義務、流程瓶頸…）且 schema 變更可追溯	1–2 週 / 2–3 週 / 3–4 週	風險：過度建模；緩解：先以 object set + 5–10 個核心查詢驅動建模（對齊 Palantir “Ontology layer serves as API” 的精神）。[119]
文件→Chunk→檢索管線	文件處理、chunk objects、media reference、keyword+vector 索引、基礎 RAG	Top-k 檢索命中率、引用可回溯；可回答 ≥20 個文件型問題	2–3 週 / 3–5 週 / 5–8 週	風險：檢索失敗導致幻覺；緩解：採用 Palantir 建議的 hybrid search、query augmentation、並把 evidence coverage 作為輸出欄位。[100]
Entity Resolution MVP	近似匹配、canonical id、merge/split 流程、人工驗證 UI	重複率下降、合併正確率、可追溯 merge 決策	2–3 週 / 3–6 週 / 6–10 週	風險：錯合併；緩解：採用 Gotham「resolution」概念做可逆合併（事件化）、加入 human-in-the-loop。[120]
風險評分與行動規劃引擎	RiskScore、Priority、ActionList、Timeline、資源估算器、審核/寫回	建議採納率、風險排序與專家一致性、行動完成率	2–4 週 / 4–8 週 / 8–12 週	風險：企業/法務風險；緩解：把每個決策與行動納入 approval + audit trail（類 AIP actions/approval）。[121]
工作台（顧問 UI）	object explorer、客戶檔案（object view）、inbox/tasks、報告模板	顧問單案交付時間下降、可重複模板數量	3–6 週 / 6–10 週 / 10–16 週	風險：UI 先行造成模型漂移；緩解：先以 schema + queries + actions 固化，再做 UI。[122]
合規與權限	ABAC/marking、敏感資料掃描、稽核報表	權限測試覆蓋率、稽核可追溯性、PDPA/GDPR 需求對應	2–4 週 / 4–8 週 / 8–16 週	風險：敏感資料外洩；緩解：mandatory control 類門控、稽核、以及資料最小化（參照 GDPR 原則）。[123]
Pilot 與硬化	測試資料集、回歸測試、自動化 CI、監控與成本控制	SLA、成本上限、模型漂移監測	3–6 週 / 6–12 週 / 12–24 週	風險：成本/效能；緩解：建立 indexing/query 成本儀表板（參照 Palantir 對 compute/volume 的管理方式）。[124]

比較、遷移成本與風險評估

與其他本體論/知識圖譜工具比較

下表聚焦「你要做顧問工作分析系統底座」時最重要的差異：是否能把本體模型直接變成可運行的工作流、是否原生支援寫回/審計/權限、以及 RDF/OWL 這類傳統本體工具鏈的推理與互通性。

面向	Palantir Ontology（Foundry/Gotham 語境）	Neo4j [111]（Property Graph）	JanusGraph（分散式 Property Graph）	GraphDB（Ontotext [125]）+ metaphactory（metaphacts [126]）	OWL/RDF（W3C [127] 規範）
資料模型	objects/properties/links + actions/functions + dynamic security（operational ontology）[128]	nodes/relationships/properties（property graph）[129]	property graph + Gremlin traversal；強調大規模與交易性 [130]	RDF triples + SPARQL；GraphDB 提供 forward-chaining/total materialization 推理 [131]	RDF 抽象資料模型；OWL 2 本體語言（classes/properties/individuals）[132]
多型/繼承	Interfaces（可 extend、多實作、link constraints）[133]	需自行以 labels/關聯設計或外掛 schema 工具	類 Neo4j，需自行設計	以 OWL/RDFS schema + 推理表達；metaphactory 提供建模/管理 UI [134]	標準化、形式化語意推理（但工程負擔較大）[135]
寫回與工作流	原生 Actions（含規模限制）、Functions、審核/稽核鏈 [136]	需自建 action service + audit log	需自建；分散式下更需嚴謹事件一致性設計	需自建；可用 SHACL/規則做驗證與推理輔助 [137]	標準偏描述與推理；操作型工作流需自建
應用層與體驗	Object Explorer / Views / Workshop / Quiver 等原生 Ontology-aware apps [122]	有可視化工具但非 Ontology+workflow 一體（需自行整合）[138]	同左（需自建應用）[139]	metaphactory 提供豐富可客製 UI 元件與快速建應用能力 [134]	標準不含 UI；需依工具組裝

面向	Palantir Ontology（Foundry/Gotham 語境）	Neo4j [111]（Property Graph）	JanusGraph（分散式 Property Graph）	GraphDB（Ontotext [125]）+ metaphactory（metaphacts [126]）	OWL/RDF（W3C [127] 規範）
權限與審計	平台級 audit logs、安全/治理、mandatory control properties 等 [140]	需自行整合 IAM/ABAC/審計	同左	GraphDB/metaphactory 具安全/治理能力但整合深度視部署而定 [141]	標準不含權限/審計
供應商鎖定	高：Ontology 與 OSDK/平台應用深耦合；OSDK 依 API name，跨環境需一致命名 [142]	中：資料與查詢語言相對可攜但仍有產品差異	中：依後端儲存（Cassandra/HBase 等）與 Gremlin 生態	中：RDF/SPARQL 標準化較佳，較易遷移，但推理/規則與工具配置仍有成本 [143]	低到中：標準化最好，但落地工程、效能與治理成本可能最高

遷移成本評估（從 Palantir 走向自建/反之）

- **Schema/命名契約成本**：OSDK 以 Ontology API name 參照 entities，官方文件指出這不同於多數 Foundry 應用使用 RID 可在部署時重寫對應；因此若你在 Palantir 上建立大量 OSDK app，未來要遷移或跨 ontology 安裝，必須維持 API names 一致或建立 mapping 策略，這是實質鎖定點。 [144]
- **寫回/審計語義成本**：Palantir 對 user edits 的一致性、offset tracking、drop-all-edits 的 schema migration 等行為若要自建，需要事件溯源（event sourcing）+ 一致性策略 + 權限門控，工程量遠高於只做讀取型 KG。 [57]
- **效能成本可見性**：Palantir 把 indexing/query/action 的 compute overhead 與 volume 作為一等公民；自建系統常見風險是「成本不可觀測」，導致規模化時爆炸。 [145]

法律/倫理/商業風險評估

資料來源合法性與隱私

- 若你的顧問系統會處理個資，需符合所在地法規。GDPR 是歐盟個資保護的法源文本；台灣則有個資法（PDPA），並已設置/規劃由個資保護委員會作為主管機關的架構。 [146]
- 若你要在 2026 前後服務歐盟市場，EU AI Act（Regulation (EU) 2024/1689）已於 2024-08-01 生效，並預期多數條文於 2026-08-02 起適用（不同類別有分階段要求）；即使你不是「高風險 AI」也可能涉及透明度、治理、文件化要求。 [147]
- 建議把「資料來源授權/使用目的/保留期限」做成 ontology objects 的 properties，並在輸出中要求 evidenceRefs，降低合規與侵權風險；這與 AIP ethics 文件強調的可審計責任鏈、人為監督相容。 [105]

IP 與內容再利用

- 你若用外部搜尋抓取內容做 RAG，需注意網站條款、著作權與資料庫權；合規做法是只保存必要片段/摘要與引用指標，而非保存整篇內容到可再散布的知識庫（此為一般工程建議）。
- 在 Palantir 生態內，文件/chunking/semantic search 的做法是把 chunk objects 與 media references 連回原始媒體，保留可追溯性，這也利於 IP 風險控管。 [104]

供應商鎖定（Vendor lock-in）

- Palantir 的強項是「平台一體化」（Ontology + Apps + Governance + AI）。同一體化也意味著遷移成本高，尤其在 OSDK 依 API name 的情況下。 144
- 若你以自建 stack 承接，建議把「Ontology schema」與「Action contracts」設計成可匯出（例如 JSON schema / OpenAPI / RDF）並持續做 schema migration 測試（對齊 Palantir branching + schema migration 的精神）。 148

可解釋性與錯誤責任

- NIST AI RMF 1.0 強調 AI 風險管理、可信任、治理與可量測；你應把模型輸出與 evidence chain、審核記錄、版本資訊視為一級資料。 149
- Palantir 的 AIP ethics 與 AI FDE 文件都把 human oversight、approval、audit trails 當成平台能力；你的顧問系統若要自動產生策略/風險預知，更應把「核可」納入流程與稽核。 94

建議的測試案例與範例輸入輸出（至少五組）

以下案例以「你自建/承接」系統的建議輸出格式示意，重點在：輸入如何映射為 objects/links、如何補足背景、如何輸出量化風險與行動清單。案例中的數值僅示意。

案例一（一句話）

輸入：
「我們的訂閱制 SaaS 最近 3 個月續約率掉到 70%，現金只能撐 6 個月，想知道該先救哪裡？」

映射到 Ontology（示意）：

- Organization: SaaS 公司
- Metric: RenewalRate（近 3 個月）、CashRunway（月）
- Risk: RevenueCollapse、RunwayShortage
- Intake: 本次顧問需求
- Links: Intake→Organization、Intake→Metric、Risk→Metric、Risk→Organization

輸出（示意）：

- RiskScore：Runway 88（P0）、Retention 82（P0）、AcquisitionEfficiency 55（P2）
- 行動清單（Top 3）：
  1) 30 天內完成「客戶分群 + 流失原因抽取」並建立 retention playbook（需求：PM 4 人天、CS 6 人天）
  2) 14 天內做成本結構重算與砍支出方案（需求：CFO 3 人天）
  3) 45 天內推出 2 個高影響續約誘因（需求：工程 10 人天、行銷 6 人天）

案例二（少量文件：財務報表 + 合約）

輸入：

- 上傳：最近 12 個月損益表、主要客戶合約 3 份（PDF）

映射（示意）：

- Documents/Chunks（含 embeddings、media references）
- Contract/Clause/Obligation objects（由抽取器建立）

- Metric：GrossMargin、CAC、AR Days
- Risk：CovenantBreach、CustomerConcentration、PaymentDelay

輸出（示意）：

- RiskScore：客戶集中度 90（P0，Evidence：合約營收占比 chunk）、現金壓力 80（P1）
- 產出「合約義務清單」：每個 obligation 連回原條款 chunk，並列出履約成本與負責人
- 里程碑：T+7 天完成合約風險矩陣、T+21 天完成替代客戶管線上線

案例三（大量文件/連結：品牌電商）

輸入：

- 連結：官網、競品 5 家、近 6 個月社群貼文、客服 FAQ、評論匯出 CSV（2 萬筆）

映射（示意）：

- Product/SKU、Channel（Shopify、IG、Marketplace）、ComplaintTheme、Policy（退貨/保固）
- Semantic search corpus：評論/客服對話切塊
- Risk：ReputationCrisis、ReturnFraud、InventoryMismatch

輸出（示意）：

- Top themes（由 object set 聚合）：尺寸不符、物流破損、客服回覆慢
- RiskScore：Reputation 78（P1）、ReturnCost 72（P1）
- 行動清單：調整尺碼表與內容、導入包材 SOP、客服 SLA（含資源與工期）

案例四（制度化階段：內控與法遵）

輸入：
「我們準備導入內控制度，想知道最小可行的 SOP/權責與稽核機制要怎麼做，避免拖垮速度。」

映射（示意）：

- Interface：InstitutionalizingStage
- Objects：Process、Control、ApprovalWorkflow、AuditEvent
- Required properties：每個流程必有 owner、輸入/輸出、風險點、控制點

輸出（示意）：

- RiskScore：目前內控缺口 85（P0）
- 建議：先做 3 條主流程（收款、採購付款、客訴退貨）與最小控制點
- 量化：每條流程控制點 ≤5、審核節點 ≤2、每週稽核抽樣 n=20

案例五（自媒體/個人品牌）

輸入：

- 一句話 + 少量文件：品牌定位簡報、過去 30 支影片連結

映射（示意）：

- Creator、AudienceSegment、ContentSeries、FunnelStep、BrandAsset（商標/視覺）

- Metric：WatchTime、CTR、EmailSignup、Conversion
- Risk：ContentDilution、IPRisk（素材/商標）

輸出（示意）：

- RiskScore：定位稀釋 68（P2）、IP/商標 75（P1）
- 行動：2 週內完成內容系列化與 CTA 統一；4 週內完成商標檢索與申請策略（需法務資源）

（完）

---

1 6 7 28 74 90 91 128 https://palantir.com/docs/foundry/ontology/overview/
https://palantir.com/docs/foundry/ontology/overview/

2 14 15 125 148 Test changes in the ontology
https://www.palantir.com/docs/foundry/ontologies/test-changes-in-ontology?utm_source=chatgpt.com

3 19 46 80 81 127 https://palantir.com/docs/foundry/ontology-sdk/overview/
https://palantir.com/docs/foundry/ontology-sdk/overview/

4 5 12 13 48 49 52 53 54 118 https://palantir.com/docs/foundry/object-backend/overview/
https://palantir.com/docs/foundry/object-backend/overview/

8 30 31 https://palantir.com/docs/foundry/ontologies/ontologies-overview/
https://palantir.com/docs/foundry/ontologies/ontologies-overview/

9 29 Ontologies · Overview
https://palantir.com/docs/foundry/ontologies/ontologies-overview/?utm_source=chatgpt.com

10 38 Interfaces · Overview
https://palantir.com/docs/foundry/interfaces/interface-overview/?utm_source=chatgpt.com

11 39 https://palantir.com/docs/foundry/interfaces/interface-overview/
https://palantir.com/docs/foundry/interfaces/interface-overview/

16 17 62 145 Usage · Ontology query compute
https://palantir.com/docs/foundry/ontologies/query-compute-usage/?utm_source=chatgpt.com

18 75 https://palantir.com/docs/foundry/api/general/overview/introduction//
https://palantir.com/docs/foundry/api/general/overview/introduction//

20 21 70 122 https://palantir.com/docs/foundry/ontology/applications/
https://palantir.com/docs/foundry/ontology/applications/

22 83 https://palantir.com/docs/gotham/api/general/overview/introduction//
https://palantir.com/docs/gotham/api/general/overview/introduction//

23 84 https://palantir.com/docs/foundry/cross-app-interactivity/enrichment-reference/
https://palantir.com/docs/foundry/cross-app-interactivity/enrichment-reference/

24 https://palantir.com/docs/foundry/aip/overview/
https://palantir.com/docs/foundry/aip/overview/

25 97 99 100 103 104 109 126 https://palantir.com/docs/foundry/ontology/ontology-augmented-generation/
https://palantir.com/docs/foundry/ontology/ontology-augmented-generation/

26 Palantir Foundry 的「Ontology」模組（微）深度研究
https://labs.botsnova.com/2025/06/05/ontology-in-palantir-foundry-quick-dive/?utm_source=chatgpt.com

27 79 https://community.palantir.com/t/openapi-spec-of-foundry-api/2943
https://community.palantir.com/t/openapi-spec-of-foundry-api/2943

32 https://palantir.com/docs/foundry/object-link-types/object-types-overview/
https://palantir.com/docs/foundry/object-link-types/object-types-overview/

33 https://palantir.com/docs/foundry/object-link-types/object-type-metadata/
https://palantir.com/docs/foundry/object-link-types/object-type-metadata/

34 44 47 Object and link types · Properties · Metadata reference
https://palantir.com/docs/foundry/object-link-types/property-metadata/?utm_source=chatgpt.com

35 Object and link types - Metadata reference
https://palantir.com/docs/foundry/object-link-types/link-type-metadata/?utm_source=chatgpt.com

36 98 https://palantir.com/docs/foundry/object-link-types/base-types/
https://palantir.com/docs/foundry/object-link-types/base-types/

37 Object and link types · Types reference
https://palantir.com/docs/foundry/object-link-types/type-reference/?utm_source=chatgpt.com

40 133 https://palantir.com/docs/foundry/interfaces/extend-interface/
https://palantir.com/docs/foundry/interfaces/extend-interface/

41 https://palantir.com/docs/foundry/object-link-types/required-properties/
https://palantir.com/docs/foundry/object-link-types/required-properties/

42 92 https://palantir.com/docs/foundry/interfaces/interface-link-types-overview/
https://palantir.com/docs/foundry/interfaces/interface-link-types-overview/

43 93 110 123 https://palantir.com/docs/foundry/object-link-types/mandatory-control-properties/
https://palantir.com/docs/foundry/object-link-types/mandatory-control-properties/

45 Create an object type
https://palantir.com/docs/foundry/object-link-types/create-object-type/?utm_source=chatgpt.com

50 https://palantir.com/docs/foundry/pipeline-builder/outputs-overview/
https://palantir.com/docs/foundry/pipeline-builder/outputs-overview/

51 Read and write unstructured files - Python
https://palantir.com/docs/foundry/transforms-python/unstructured-files/?utm_source=chatgpt.com

55 56 57 58 59 108 136 https://palantir.com/docs/foundry/object-edits/how-edits-applied/
https://palantir.com/docs/foundry/object-edits/how-edits-applied/

60 124 https://palantir.com/docs/foundry/ontologies/compute-usage/
https://palantir.com/docs/foundry/ontologies/compute-usage/

61 102 https://palantir.com/docs/foundry/ontologies/volume-usage/
https://palantir.com/docs/foundry/ontologies/volume-usage/

63 https://palantir.com/docs/foundry/action-types/scale-property-limits/
https://palantir.com/docs/foundry/action-types/scale-property-limits/

64 https://palantir.com/docs/foundry/object-indexing/funnel-streaming-pipelines/
https://palantir.com/docs/foundry/object-indexing/funnel-streaming-pipelines/

65 66 https://www.palantir.com/docs/foundry/object-indexing/data-restrictions
https://www.palantir.com/docs/foundry/object-indexing/data-restrictions

67 95 120 https://palantir.com/docs/gotham/api/revdb-resources/resolution/resolution-basics//
https://palantir.com/docs/gotham/api/revdb-resources/resolution/resolution-basics//

68 https://www.palantir.com/foundry-entity-resolution/
https://www.palantir.com/foundry-entity-resolution/

69 https://palantir.com/docs/foundry/code-examples/fuzzy-matching-transforms/
https://palantir.com/docs/foundry/code-examples/fuzzy-matching-transforms/

71 https://palantir.com/docs/foundry/object-explorer/overview/
https://palantir.com/docs/foundry/object-explorer/overview/

72 https://palantir.com/docs/foundry/workshop/overview/
https://palantir.com/docs/foundry/workshop/overview/

73 https://palantir.com/docs/foundry/quiver/overview/
https://palantir.com/docs/foundry/quiver/overview/

76 77 https://pypi.org/project/foundry-platform-sdk/0.33.0/
https://pypi.org/project/foundry-platform-sdk/0.33.0/

78 https://github.com/palantir/foundry-platform-python
https://github.com/palantir/foundry-platform-python

82 https://palantir.com/docs/foundry/ontology-sdk/typescript-subscriptions/
https://palantir.com/docs/foundry/ontology-sdk/typescript-subscriptions/

85 https://palantir.com/docs/foundry/object-link-types/enable-gotham-integration/
https://palantir.com/docs/foundry/object-link-types/enable-gotham-integration/

86 Overview · Security
https://palantir.com/docs/foundry/security/overview/?utm_source=chatgpt.com

87 96 101 140 Security auditing · Audit logs
https://palantir.com/docs/foundry/security/audit-logs-overview/?utm_source=chatgpt.com

88 https://www.palantir.com/docs/foundry/ai-fde/security-and-governance/
https://www.palantir.com/docs/foundry/ai-fde/security-and-governance/

89 94 105 106 121 https://palantir.com/docs/foundry/aip/ethics-governance/
https://palantir.com/docs/foundry/aip/ethics-governance/

107 149 https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf
https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf

111 142 144 https://palantir.com/docs/foundry/ontology-sdk/marketplace-osdk-deployment/
https://palantir.com/docs/foundry/ontology-sdk/marketplace-osdk-deployment/

112 116 129 138 https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/
https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/

113 130 https://github.com/JanusGraph/janusgraph
https://github.com/JanusGraph/janusgraph

114 132 143 https://www.w3.org/TR/rdf12-concepts/
https://www.w3.org/TR/rdf12-concepts/

115 131 137 https://graphdb.ontotext.com/documentation/11.3/reasoning.html
https://graphdb.ontotext.com/documentation/11.3/reasoning.html

117 https://palantir.com/docs/foundry/object-explorer/search-syntax/
https://palantir.com/docs/foundry/object-explorer/search-syntax/

119 Recommended Project and team structure
https://palantir.com/docs/foundry/building-pipelines/recommended-project-structure/?utm_source=chatgpt.com

134 https://www.semantic-web-journal.net/system/files/swj2051.pdf
https://www.semantic-web-journal.net/system/files/swj2051.pdf

135 https://www.w3.org/TR/owl2-overview/
https://www.w3.org/TR/owl2-overview/

139 https://tinkerpop.apache.org/gremlin.html
https://tinkerpop.apache.org/gremlin.html

141 https://resourcehub.metaphacts.net/resource/Help%3ADataAccessControl
https://resourcehub.metaphacts.net/resource/Help%3ADataAccessControl

146 https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng
https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng

147 https://commission.europa.eu/news-and-media/news/ai-act-enters-force-2024-08-01_en
https://commission.europa.eu/news-and-media/news/ai-act-enters-force-2024-08-01_en